Lab08VitorAugustoCosta267117

```r
library(Replicate)
```

```r
reticulate::py_require("polars")
```

```r
import polars as pl
```

```r
# 1. Mapeamento Lazy dos arquivos
# quote_char=None evita erro com aspas não fechadas nos títulos
basics_lf = pl.scan_csv(
    "title.basics0.tsv.gz",
    separator="\t",
    null_values=r"\N",
    quote_char=None
)

ratings_lf = pl.scan_csv(
    "title.ratings.tsv.gz",
    separator="\t",
    null_values=r"\N",
    quote_char=None
)

principals_lf = pl.scan_csv(
    "title.principals0.tsv.gz",
    separator="\t",
    null_values=r"\N",
    quote_char=None
)


# Questão 1: 5 filmes com maiores notas (desempate: numVotes)
```

```
q1 = (
    basics_lf
    .filter(pl.col("titleType") == "movie")
    .select(["tconst", "primaryTitle"])
    .join(ratings_lf, on="tconst", how="inner")
    .sort(by=["averageRating", "numVotes"], descending=[True, True])
    .select(["tconst", "primaryTitle", "averageRating", "numVotes"])
    .head(5)
    .collect() # <- Executa o plano e carrega apenas o resultado final na RAM
)

print("1. Top 5 Filmes:")
```

```
1. Top 5 Filmes:
```

```
print(q1)
```

```
shape: (5, 4)

 tconst      primaryTitle               averageRating  numVotes
 ---         ---                        ---            ---
 str         str                        f64            i64

 tt33075815  Kaveri                     10.0           1023
 tt27815015  Kurukku                    10.0           451
 tt28450376  Jedal Dar Omghe 30 Metri   10.0           142
 tt27859713  Sargashte                  10.0           134
 tt33507675  Gorgeous Rascal            10.0           115
```

```
# Questão 2: Gênero mais frequente (filmes com nota > 8)

q2 = (
    basics_lf
    .filter(pl.col("titleType") == "movie")
    .select(["tconst", "genres"])
    .join(
        ratings_lf.filter(pl.col("averageRating") > 8.0).select("tconst"),
        on="tconst",
        how="inner"
```

```
    )
    .drop_nulls("genres")
    .with_columns(pl.col("genres").str.split(","))
    .explode("genres")
    .with_columns(pl.col("genres").str.strip_chars())
    .group_by("genres")
    .agg(pl.len().alias("frequencia"))
    .sort("frequencia", descending=True)
    .head(1)
    .collect()
)

print("\n2. Gênero mais frequente (Nota > 8):")
```

```
2. Gênero mais frequente (Nota > 8):
```

```
print(q2)
```

```
shape: (1, 2)

 genres        frequencia
 ---           ---
 str           u32

 Documentary   10564
```

```
# Questão 3: Top 3 atores/atrizes
# Isolamos os IDs dos filmes > 7.5 para cruzar com o elenco
filmes_75_lf = (
    basics_lf
    .filter(pl.col("titleType") == "movie")
    .select("tconst")
    .join(
        ratings_lf.filter(pl.col("averageRating") > 7.5).select("tconst"),
        on="tconst",
        how="inner"
    )
)
```

```python
q3 = (
    principals_lf
    .filter(pl.col("category").is_in(["actor", "actress"]))
    .select(["tconst", "nconst"])
    .join(filmes_75_lf, on="tconst", how="inner")
    .group_by("nconst")
    .agg(pl.col("tconst").n_unique().alias("qtd_filmes")) # Conta filmes distintos por ator
    .sort("qtd_filmes", descending=True)
    .head(3)
    .collect()
)

print("\n3. Top 3 Atores/Atrizes (Nota > 7.5):")
```

```
3. Top 3 Atores/Atrizes (Nota > 7.5):
```

```python
print(q3)
```

```
shape: (3, 2)

 nconst      qtd_filmes
 ---         ---
 str         u32

 nm0004660   199
 nm0595934   155
 nm3183374   124
```

horario

```python
Sys.setenv(TZ = "America/Sao_Paulo")
Sys.time()
```

```
[1] "2026-10-14 14:02:55 -03"
```